PML Assignment 3: Logistic Regression vs. SVM on Breast Cancer Wisconsin Data

Brian Butler

March 2026

1 Overview

We compare **Logistic Regression**, **Linear SVM**, and **RBF-kernel SVM** on the **Breast Cancer Wisconsin (Diagnostic)** dataset (n=569 samples, d=30 features, binary: malignant vs. benign). All models are wrapped in `Pipeline(StandardScaler, model)` so that feature standardization is performed *inside* cross-validation, preventing data leakage. Hyperparameters are tuned via `GridSearchCV` with 5-fold `StratifiedKFold`, and a single held-out test set (20%) is evaluated once at the end.

2 Experimental Setup

2.1 Data Split

An 80/20 stratified split (`random_state=42`) produces 455 training and 114 test samples. The test set is never used during tuning.

2.2 Pipeline & CV Configuration

```
# Pipeline prevents data leakage: scaling fit on each CV fold
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("clf",    model)   # LR / LinearSVC / SVC
])
# Stratified 5-fold CV ensures class balance in every fold
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
gs = GridSearchCV(pipe, param_grid, cv=cv, scoring="accuracy")
```

2.3 Hyperparameter Grids

Table 1: Search grids for each model

Model	Parameter(s)	Grid Values
Logistic Regression	C	$\{0.001, 0.01, 0.1, 1, 10, 100\}$
Linear SVC	C	$\{0.001, 0.01, 0.1, 1, 10, 100\}$
RBF SVC	C	$\{0.01, 0.1, 1, 10, 100\}$
	γ	$\{0.001, 0.01, 0.1, 1\}$

Note: `LinearSVC` lacks `predict_proba`, so it is wrapped in `CalibratedClassifierCV` (3-fold internal CV) to enable ROC–AUC computation.

3 Results

3.1 Best CV Hyperparameters

Table 2: Best hyperparameters selected by 5-fold GridSearchCV

Model	Best Params	Best CV Accuracy
Logistic Regression	$C = 0.1$	0.9824
Linear SVC	$C = 0.01$	0.9780
RBF SVC	$C = 10,\ \gamma = 0.01$	0.9758

All three models achieve CV accuracies above 97.5%, reflecting that the breast cancer dataset is well-separated after standardization.

3.2 Held-Out Test-Set Performance

Table 3: Test-set metrics (114 samples, evaluated once)

Model	Test Accuracy	ROC–AUC	Confusion Matrix
Logistic Regression	0.9737	0.9957	$\begin{pmatrix} 40 & 2 \\ 1 & 71 \end{pmatrix}$
Linear SVC	0.9825	0.9960	$\begin{pmatrix} 41 & 1 \\ 1 & 71 \end{pmatrix}$
RBF SVC	0.9825	0.9977	$\begin{pmatrix} 41 & 1 \\ 1 & 71 \end{pmatrix}$

Confusion matrix format: rows = actual (malignant, benign), columns = predicted. TN = true negatives (malignant correctly identified); TP = true positives (benign correctly identified).

4 Discussion

4.1 Model Comparison

All three classifiers achieve excellent performance ($> 97\%$ accuracy, $> 99.5\%$ AUC), consistent with existing literature on this dataset.

Logistic Regression achieves the highest CV accuracy (0.9824) but slightly lower test accuracy (0.9737) with 2 false positives versus 1 for the SVMs. The selected C=0.1 indicates stronger regularization is preferred, suggesting the decision boundary benefits from shrinkage.

Linear SVC matches RBF SVC on test accuracy (0.9825) with a very small C=0.01, indicating that strong regularization of the linear margin yields a well-generalizing model.

RBF SVC achieves the best ROC–AUC (0.9977) with C=10 and γ=0.01. The moderate γ keeps the kernel bandwidth wide enough to avoid overfitting, while the larger C allows a tighter margin fit.

4.2 Linear vs. Nonlinear Boundaries

The comparable performance of linear models (Logistic, LinearSVC) to RBF SVC suggests the two classes are *nearly linearly separable* in the 30-dimensional feature space. The slight AUC advantage of RBF SVC indicates the nonlinear kernel captures subtle boundary curvature that linear models miss, though this translates to identical hard-label accuracy (0.9825).

4.3 Regularization Insights

All models favor moderate-to-strong regularization: Logistic C=0.1, LinearSVC C=0.01, RBF SVC C=10 (moderate given the wider kernel). This is expected: with only 455 training samples across 30 features, regularization prevents overfitting to noise.

5 Reproducibility

```
SEED      = 42            # Fixes all random states
TEST_SIZE = 0.20          # 80/20 stratified split
```

```
N_FOLDS   = 5             # StratifiedKFold(shuffle=True, seed=42)
# Pipeline: StandardScaler -> model (fit inside each CV fold)
# GridSearchCV: scoring="accuracy", refit=True, n_jobs=-1
```

Three sources of randomness are fixed: (1) `random_state=42` in `train_test_split` ensures identical partitions; (2) `random_state=42` in `StratifiedKFold` ensures identical folds; (3) `random_state=42` in each model ensures deterministic optimization.

6 Appendix: Full Source Code

```
"""
PML Assignment 3 - Logistic Regression vs SVM
Breast Cancer Wisconsin Data
"""
import numpy as np, pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import (train_test_split,
    GridSearchCV, StratifiedKFold)
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC, SVC
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import (accuracy_score, confusion_matrix,
    roc_auc_score)
import warnings; warnings.filterwarnings("ignore")

SEED, TEST_SIZE, N_FOLDS = 42, 0.20, 5
C_GRID     = [0.001, 0.01, 0.1, 1, 10, 100]
GAMMA_GRID = [0.001, 0.01, 0.1, 1]

data = load_breast_cancer()
X, y = data.data, data.target

# 80/20 stratified split; test set used once at the end
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, random_state=SEED, stratify=y)

cv = StratifiedKFold(n_splits=N_FOLDS, shuffle=True,
                     random_state=SEED)

def tune(name, pipe, grid):
    """Run GridSearchCV and print best result."""
    gs = GridSearchCV(pipe, grid, cv=cv, scoring="accuracy",
                      refit=True, n_jobs=-1)
    gs.fit(X_train, y_train)
    print(f"{name}: best_cv={gs.best_score_:.4f} "
          f"params={gs.best_params_}")
    return gs

# Model 1: Logistic Regression
gs_lr = tune("LR", Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=10000,
                               random_state=SEED))
]), {"clf__C": C_GRID})

# Model 2: Linear SVC (calibrated for predict_proba)
gs_lsvc = tune("LSVC", Pipeline([
    ("scaler", StandardScaler()),
    ("clf", CalibratedClassifierCV(
        LinearSVC(max_iter=50000, dual="auto",
                  random_state=SEED), cv=3))
]), {"clf__estimator__C": C_GRID})

# Model 3: RBF SVC
gs_rbf = tune("RBF", Pipeline([
    ("scaler", StandardScaler()),
    ("clf", SVC(kernel="rbf", probability=True,
                random_state=SEED))
]), {"clf__C": [0.01,0.1,1,10,100],
     "clf__gamma": GAMMA_GRID})

# Final test evaluation (single pass)
for name, gs in [("LR",gs_lr), ("LSVC",gs_lsvc),
                 ("RBF",gs_rbf)]:
    yp = gs.predict(X_test)
```

```
    yprob = gs.predict_proba(X_test)[:,1]
    print(f"{name}: acc={accuracy_score(y_test,yp):.4f}"
          f" auc={roc_auc_score(y_test,yprob):.4f}")
    print(confusion_matrix(y_test, yp))
```